

Experiment 5: Hadoop Installation, Configuration, and HDFS Operations

Mapped CO(s): CO2

Aim:

To install and configure Hadoop in a single-node environment and perform basic HDFS operations using command-line tools.

Objectives:

1. To understand the Hadoop ecosystem and HDFS architecture.
2. To install and configure Hadoop environment variables and XML configuration files.
3. To start Hadoop core daemons (NameNode, DataNode, SecondaryNameNode).
4. To perform basic HDFS file operations (creating directories, uploading, reading, copying, downloading, and deleting files).
5. To verify the successful execution of HDFS commands and monitor cluster capacity.

Prerequisites:

- Basic knowledge of Linux shell commands.
- Java Development Kit (JDK 8/11) installed and configured.
- Ubuntu / Linux operating system environment.
- Apache Hadoop software package (Hadoop 3.4.2).
- Terminal / Bash command-line interface.

Theory/Background:

Apache Hadoop is an open-source distributed storage and processing framework designed to handle massive amounts of structured and unstructured data across commodity hardware clusters.

The main components of the Apache Hadoop ecosystem include:

- **HDFS (Hadoop Distributed File System):** A distributed, scalable, and portable file system designed with a master-slave architecture (**NameNode** and **DataNodes**) that stores data across nodes in blocks with configurable replication for high fault tolerance.
- **MapReduce:** A software framework and programming model for processing vast amounts of data in parallel across Hadoop clusters.
- **YARN (Yet Another Resource Negotiator):** The cluster management layer that schedules jobs and allocates system compute resources efficiently.

- **Hadoop Common:** The fundamental utilities, libraries, and Java classes required by all Hadoop submodules.

In this experiment, we configure a single-node pseudo-distributed Hadoop environment and execute fundamental HDFS file system management commands.

Step 1: Verify Hadoop Environment Variables and Installation

We check that the Hadoop home path (`$HADOOP_HOME`), configuration directory (`$HADOOP_CONF_DIR`), and the `hadoop` binary executable path are set correctly in `~/ .bashrc` , and check the Hadoop version.

In [1]: *# Verify Hadoop Environment Variables and Binary*

```
!echo "HADOOP_HOME: $HADOOP_HOME"
!echo "HADOOP_CONF_DIR: $HADOOP_CONF_DIR"
!which hadoop
!hadoop version
```

HADOOP_HOME: /home/jeetumehra/PYPrograms/hadoop-3.4.2

HADOOP_CONF_DIR: /home/jeetumehra/PYPrograms/hadoop-3.4.2/etc/hadoop

/home/jeetumehra/PYPrograms/hadoop-3.4.2/bin/hadoop

Hadoop 3.4.2

Source code repository <https://github.com/apache/hadoop.git> -r 84e8b89ee2ebe6923691205b9e171badde7a495c

Compiled by ahmarsu on 2025-08-20T10:30Z

Compiled on platform linux-x86_64

Compiled with protoc 3.23.4

From source with checksum fa94c67d4b4be021b9e9515c9b0f7b6

This command was run using /home/jeetumehra/PYPrograms/hadoop-3.4.2/share/hadoop/common/hadoop-common-3.4.2.jar

Step 2: Configure Hadoop Core and HDFS XML Files

We configure:

1. `core-site.xml` : Defines the default file system URI (`fs.defaultFS` set to `hdfs://localhost:9000`).
2. `hdfs-site.xml` : Defines replication factor (`dfs.replication` set to `1` for single-node setup) and local filesystem paths for NameNode and DataNode metadata and blocks.

In [2]: *# Display core-site.xml configuration*

```
!cat ~/PYPrograms/hadoop-3.4.2/etc/hadoop/core-site.xml
```

```
<?xml version="1.0"?>
<?xml-stylesheet type="text/xsl" href="configuration.xsl"?>

<configuration>

  <property>
    <name>fs.defaultFS</name>
    <value>hdfs://localhost:9000</value>
  </property>

</configuration>
```

In [3]: *# Display hdfs-site.xml configuration*
!cat ~/PYPrograms/hadoop-3.4.2/etc/hadoop/hdfs-site.xml

```
<?xml version="1.0"?>
<?xml-stylesheet type="text/xsl" href="configuration.xsl"?>

<configuration>

  <property>
    <name>dfs.replication</name>
    <value>1</value>
  </property>

  <property>
    <name>dfs.namenode.name.dir</name>
    <value>file:///home/jeetumehra/hadoop_data/namenode</value>
  </property>

  <property>
    <name>dfs.datanode.data.dir</name>
    <value>file:///home/jeetumehra/hadoop_data/datanode</value>
  </property>

</configuration>
```

Step 3: Configure MapReduce and YARN Configuration Files

We configure:

1. `mapred-site.xml` : Specifies the MapReduce execution framework (set to `yarn`).
2. `yarn-site.xml` : Specifies YARN NodeManager auxiliary services (set to `mapreduce_shuffle`).

In [4]: *# Display mapred-site.xml and yarn-site.xml configurations*
!cat ~/PYPrograms/hadoop-3.4.2/etc/hadoop/mapred-site.xml
!cat ~/PYPrograms/hadoop-3.4.2/etc/hadoop/yarn-site.xml

```

<?xml version="1.0"?>
<?xml-stylesheet type="text/xsl" href="configuration.xsl"?>

<configuration>

    <property>
        <name>mapreduce.framework.name</name>
        <value>yarn</value>
    </property>

</configuration>
<?xml version="1.0"?>
<?xml-stylesheet type="text/xsl" href="configuration.xsl"?>

<configuration>

    <property>
        <name>yarn.nodemanager.aux-services</name>
        <value>mapreduce_shuffle</value>
    </property>

</configuration>

```

Step 4: Create Hadoop Data Storage Directories

We create directory structures for NameNode and DataNode data persistence and verify their permissions.

```

In [5]: # Create NameNode and DataNode directories
!mkdir -p ~/hadoop_data/namenode
!mkdir -p ~/hadoop_data/datanode
!ls -l ~/hadoop_data

```

```

total 8
drwxr-xr-x 2 jeetumehra jeetumehra 4096 Sep 17 14:17 datanode
drwxr-xr-x 2 jeetumehra jeetumehra 4096 Sep 17 14:17 namenode

```

Step 5: Start Hadoop DFS Daemons and Verify with JPS

We start the Hadoop distributed file system services using `start-dfs.sh` and verify that all core Java daemons (`NameNode` , `DataNode` , `SecondaryNameNode`) are actively running using the `jps` tool.

```

In [6]: # Start Hadoop DFS services and check Java processes
!start-dfs.sh
!jps

```

```

Starting namenodes on [localhost]
Starting datanodes
Starting secondary namenodes [JeetuMehra]
JeetuMehra: Warning: Permanently added 'jeetumehra' (ED25519) to the list of known hosts.
2026-09-17 14:19:30,721 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
30600 DataNode
30874 SecondaryNameNode
30444 NameNode
31055 Jps

```

Step 6: Create User Directory in HDFS and Verify

We create a base working directory for the user in HDFS using `hdfs dfs -mkdir -p` and list the parent directory using `hdfs dfs -ls`.

```

In [7]: # Create directory in HDFS and verify
!hdfs dfs -mkdir -p /user/$USER/hadoop_exp5
!hdfs dfs -ls /user/$USER

```

```

2026-09-17 14:20:01,039 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
2026-09-17 14:20:07,067 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
Found 1 items
drwxr-xr-x - jeetumehra supergroup      0 2026-09-17 14:20 /user/jeetumehra/hadoop_exp5

```

Step 7: Create a Local Sample File and Upload to HDFS

We create a local text file `sample.txt`, inspect its content, upload it to the HDFS directory using `hdfs dfs -put`, and confirm its presence using `hdfs dfs -ls`.

```

In [8]: # Create local sample file and upload to HDFS
!echo -e "Hadoop HDFS Experiment 5\nNameNode manages metadata\nDataNode stores data"
!cat ~/sample.txt
!hdfs dfs -put ~/sample.txt /user/$USER/hadoop_exp5/
!hdfs dfs -ls /user/$USER/hadoop_exp5

```

```

Hadoop HDFS Experiment 5
NameNode manages metadata
DataNode stores data blocks
HDFS provides distributed storage
2026-09-17 14:21:07,416 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
2026-09-17 14:21:12,826 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
Found 1 items
-rw-r--r-- 1 jeetumehra supergroup    113 2026-09-17 14:21 /user/jeetumehra/hadoop_exp5/sample.txt

```

Step 8: Display File Contents from HDFS and Copy within HDFS

We read the content of the file directly from HDFS using `hdfs dfs -cat` and create a copy inside the HDFS directory using `hdfs dfs -cp`.

```
In [9]: # Display file content and copy file within HDFS
!hdfs dfs -cat /user/$USER/hadoop_exp5/sample.txt
!hdfs dfs -cp /user/$USER/hadoop_exp5/sample.txt /user/$USER/hadoop_exp5/sample_cop
!hdfs dfs -ls /user/$USER/hadoop_exp5
```

```
2026-09-17 14:21:36,988 WARN util.NativeCodeLoader: Unable to load native-hadoop lib
rary for your platform... using builtin-java classes where applicable
Hadoop HDFS Experiment 5
NameNode manages metadata
DataNode stores data blocks
HDFS provides distributed storage
2026-09-17 14:22:01,318 WARN util.NativeCodeLoader: Unable to load native-hadoop lib
rary for your platform... using builtin-java classes where applicable
2026-09-17 14:22:06,923 WARN util.NativeCodeLoader: Unable to load native-hadoop lib
rary for your platform... using builtin-java classes where applicable
Found 2 items
-rw-r--r--  1 jeetumehra supergroup      113 2026-09-17 14:21 /user/jeetumehra/ha
doop_exp5/sample.txt
-rw-r--r--  1 jeetumehra supergroup      113 2026-09-17 14:22 /user/jeetumehra/ha
doop_exp5/sample_copy.txt
```

Step 9: Download File from HDFS to Local File System and Verify

We use `hdfs dfs -get` to download `sample_copy.txt` from HDFS to the local user directory as `sample_download.txt` and verify its contents.

```
In [10]: # Download file from HDFS and display local file content
!hdfs dfs -get /user/$USER/hadoop_exp5/sample_copy.txt ~/sample_download.txt
!cat ~/sample_download.txt
```

```
2026-09-17 14:22:28,316 WARN util.NativeCodeLoader: Unable to load native-hadoop lib
rary for your platform... using builtin-java classes where applicable
Hadoop HDFS Experiment 5
NameNode manages metadata
DataNode stores data blocks
HDFS provides distributed storage
```

Step 10: Check HDFS Disk Space and Delete File from HDFS

We check overall HDFS storage capacity using `hdfs dfs -df -h` and delete `sample_copy.txt` from HDFS using `hdfs dfs -rm`.

```
In [11]: # Check HDFS disk free space and remove file
!hdfs dfs -df -h
!hdfs dfs -rm /user/$USER/hadoop_exp5/sample_copy.txt
!hdfs dfs -ls /user/$USER/hadoop_exp5
```

```
2026-09-17 14:22:51,690 WARN util.NativeCodeLoader: Unable to load native-hadoop lib
ary for your platform... using builtin-java classes where applicable
Filesystem              Size      Used Available Use%
hdfs://localhost:9000  1006.9 G   24.2 K    905.4 G    0%
2026-09-17 14:23:23,098 WARN util.NativeCodeLoader: Unable to load native-hadoop lib
ary for your platform... using builtin-java classes where applicable
Deleted /user/jeetumehra/hadoop_exp5/sample_copy.txt
2026-09-17 14:23:27,428 WARN util.NativeCodeLoader: Unable to load native-hadoop lib
ary for your platform... using builtin-java classes where applicable
Found 1 items
-rw-r--r--    1 jeetumehra supergroup          113 2026-09-17 14:21 /user/jeetumehra/ha
doop_exp5/sample.txt
```

Exercises/Assignments

Exercise 1: Create a new directory in HDFS and upload multiple files

We create two new text files locally (`file1.txt` and `file2.txt`), create a new HDFS directory `/user/$USER/hadoop_exp5_multiple` , upload both files simultaneously using `hdfs dfs -put` , and verify with `hdfs dfs -ls` .

```
In [12]: # Create two local files, create target HDFS directory, upload both files and list
!echo "First additional Hadoop file" > ~/file1.txt
!echo "Second additional Hadoop file" > ~/file2.txt
!hdfs dfs -mkdir -p /user/$USER/hadoop_exp5_multiple
!hdfs dfs -put ~/file1.txt ~/file2.txt /user/$USER/hadoop_exp5_multiple/
!hdfs dfs -ls /user/$USER/hadoop_exp5_multiple
```

```

2026-09-17 14:35:34,688 WARN util.NativeCodeLoader: Unable to load native-hadoop lib
rary for your platform... using builtin-java classes where applicable
2026-09-17 14:35:35,960 WARN util.NativeCodeLoader: Unable to load native-hadoop lib
rary for your platform... using builtin-java classes where applicable
2026-09-17 14:35:37,619 WARN util.NativeCodeLoader: Unable to load native-hadoop lib
rary for your platform... using builtin-java classes where applicable
Found 2 items
-rw-r--r--  1 jeetumehra supergroup          0 2026-09-17 14:35 /user/jeetumehra/ha
doop_exp5_multiple/file1.txt
-rw-r--r--  1 jeetumehra supergroup        30 2026-09-17 14:35 /user/jeetumehra/ha
doop_exp5_multiple/file2.txt

```

Exercise 2: Copy a file from one HDFS directory to another

We create a new destination directory `/user/$USER/hadoop_exp5_copy` in HDFS, copy `file1.txt` from `hadoop_exp5_multiple` to `hadoop_exp5_copy` using `hdfs dfs -cp`, and list the target directory.

```

In [13]: # Create destination directory in HDFS, copy file from another HDFS directory, and
!hdfs dfs -mkdir -p /user/$USER/hadoop_exp5_copy
!hdfs dfs -cp /user/$USER/hadoop_exp5_multiple/file1.txt /user/$USER/hadoop_exp5_co
!hdfs dfs -ls /user/$USER/hadoop_exp5_copy

```

```

2026-09-17 14:36:05,965 WARN util.NativeCodeLoader: Unable to load native-hadoop lib
rary for your platform... using builtin-java classes where applicable
2026-09-17 14:36:06,974 WARN util.NativeCodeLoader: Unable to load native-hadoop lib
rary for your platform... using builtin-java classes where applicable
2026-09-17 14:36:08,111 WARN util.NativeCodeLoader: Unable to load native-hadoop lib
rary for your platform... using builtin-java classes where applicable
Found 1 items
-rw-r--r--  1 jeetumehra supergroup          0 2026-09-17 14:36 /user/jeetumehra/ha
doop_exp5_copy/file1.txt

```

Exercise 3: Delete a directory from HDFS

We delete the directory `/user/$USER/hadoop_exp5_copy` and its contents recursively using `hdfs dfs -rm -r` and list `/user/$USER` to confirm removal.

```

In [14]: # Remove HDFS directory recursively and List parent directory
!hdfs dfs -rm -r /user/$USER/hadoop_exp5_copy
!hdfs dfs -ls /user/$USER

```

```

2026-09-17 14:36:27,748 WARN util.NativeCodeLoader: Unable to load native-hadoop lib
rary for your platform... using builtin-java classes where applicable
Deleted /user/jeetumehra/hadoop_exp5_copy
2026-09-17 14:36:28,826 WARN util.NativeCodeLoader: Unable to load native-hadoop lib
rary for your platform... using builtin-java classes where applicable
Found 2 items
drwxr-xr-x  - jeetumehra supergroup          0 2026-09-17 14:23 /user/jeetumehra/ha
doop_exp5
drwxr-xr-x  - jeetumehra supergroup          0 2026-09-17 14:35 /user/jeetumehra/ha
doop_exp5_multiple

```

Exercise 4: Display the contents of an HDFS file

We display the contents of the file `file2.txt` stored in HDFS using `hdfs dfs -cat`.

```
In [15]: # Display content of HDFS file
!hdfs dfs -cat /user/$USER/hadoop_exp5_multiple/file2.txt
```

2026-09-17 14:37:10,124 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
Second additional Hadoop file

Exercise 5: Check the available space and disk usage in HDFS

We inspect the cluster-wide storage capacity using `hdfs dfs -df -h` and check the space consumed by files under the user's directory using `hdfs dfs -du -h`.

```
In [16]: # Check cluster disk capacity and directory space consumption
!hdfs dfs -df -h
!hdfs dfs -du -h /user/$USER
```

2026-09-17 14:37:45,392 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable

Filesystem	Size	Used	Available	Use%
hdfs://localhost:9000	1006.9 G	24.2 K	905.4 G	0%

2026-09-17 14:38:02,118 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable

```
113 /user/jeetumehra/hadoop_exp5
30 /user/jeetumehra/hadoop_exp5_multiple
```

Experimental Results & Observations

Observations:

1. Hadoop 3.4.2 was successfully installed, environment variables configured, and core XML parameters (`fs.defaultFS` , `dfs.replication` , `dfs.namenode.name.dir` , `dfs.datanode.data.dir`) mapped.
2. The core Hadoop services (`NameNode` , `DataNode` , `SecondaryNameNode`) were successfully started and verified via `jps`.
3. HDFS directories were created seamlessly using `hdfs dfs -mkdir -p`.
4. File operations including upload (`-put`), listing (`-ls`), viewing (`-cat`), copying (`-cp`), downloading (`-get`), disk space checks (`-df`), and deletion (`-rm` , `-rm -r`) were executed with complete verification.

Result:

Hadoop was successfully installed and configured in a single-node pseudo-distributed environment, and all fundamental HDFS file management operations and exercises were executed without errors.

Conclusion:

This experiment demonstrated the end-to-end architecture, configuration, and practical command-line operations of the Hadoop Distributed File System (HDFS). The hands-on execution provided clear understanding of distributed file storage, block management, and command semantics in Apache Hadoop.